



TOPAZ high frequency river forcing from GloFAS data: technical documentation

A. ACCHIARDI
adrien.acchiardi@ensta.fr

Contents

Introduction	1
1 Setup and requirements	2
1.1 Python environment setup	2
1.2 Required files	3
1.3 Required data	3
2 How to compute high frequency river forcing	4
2.1 Common issues	4
3 Computing the estuaries edit file: how to change the position of an estuary	5
4 Computing the GloFAS correction file	8
5 Details on the program	9

Introduction

This document explains how to compute the high frequency river forcing `.[ab]` (6-hourly) file from GloFAS data.

The different sections cover the different steps. Section [1](#) covers how to setup the Python environment to use the programs, as well as the necessary files. Section [2](#) explains the steps to compute the river forcing `.[ab]` file. Section [3](#) describes how to use the graphical interface to change the position of an estuary. Section [4](#) explains how to compute the correction file applied when computing the river forcing.

What if the program doesn't work ? If something goes wrong in the program, don't panic, it's probably easy to fix. Section [2](#) contains a list of common issues that may help you resolve the problem. If that doesn't help, section [5](#) gives details on how the program is built and could help with problems that may appear. In general read the error code, and ask somebody who uses Python often. Using a debugger you'll be able to find the line that's causing a problem; and fix it from there.

1 Setup and requirements

1.1 Python environment setup

Certain libraries are required to execute the programs, the requirements change depending on which program you want to use. Here are the necessary environments for the different functionalities.

To compute river forcing file only:

- Python (version 3.10 or above)
- Numpy (version 2.1.3 or above)
- Pandas (version 2.2.3)
- xarray (version 2024.11.0)
- scipy (version 1.15.2)
- scikit-image (version 0.25.2)

To move an estuary with the graphical interface:

- Python (version 3.10 or above)
- Numpy (version 2.1.3 or above)
- Pandas (version 2.2.3)
- xarray (version 2024.11.0)
- scipy (version 1.15.2)
- scikit-image (version 0.25.2)
- matplotlib (version 3.10.0)
- pyqtgraph (version 0.13.7)
- cartopy (version 0.24.1)

To compute GloFAS correction files only

- Python (version 3.10 or above)
- Numpy (version 2.1.3 or above)
- Pandas (version 2.2.3)
- xarray (version 2024.11.0)
- scipy (version 1.15.2)

It is recommended to create a virtual anaconda environment, and to install these packages using: `conda install <package>=<version>`

1.2 Required files

The program is contained in the folder named TOPAZ runoff. The architecture of this folder with the essential files is described on Figure 1

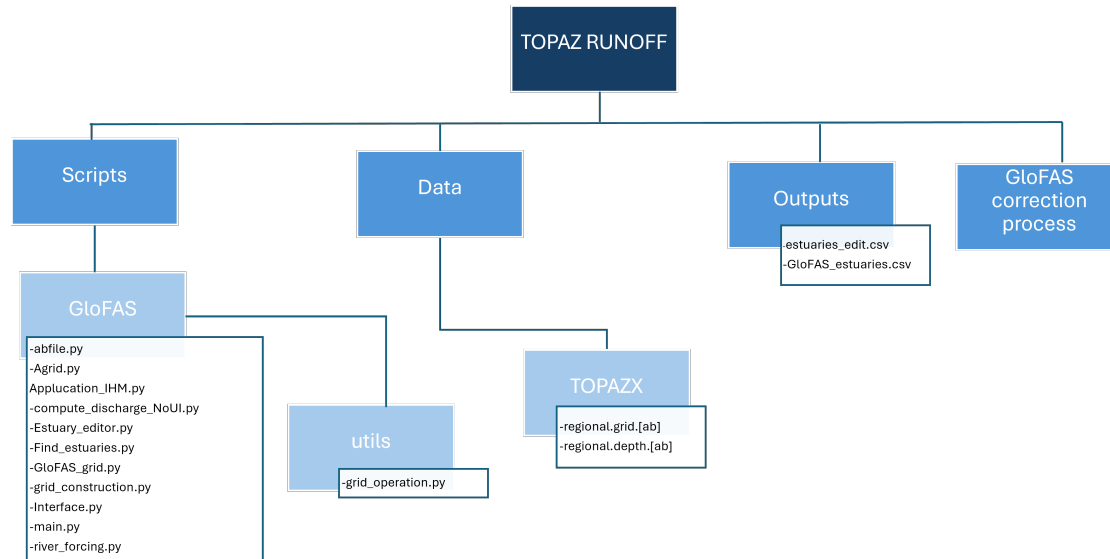


Figure 1 : Architecture of TOPAZ runoff folder

1.3 Required data

Before being able to launch the scripts, certain data files are necessary:

- The **TOPAZ grid A file** is the regional grid .a file for the version of TOPAZ that is being used. The corresponding .b file also needs to be in the same directory
- the **TOPAZ depth grid A file** is the depth .a file, it contains the depth of the points onto which the estuaries will be interpolated. The corresponding .b file also needs to be in the same directory
- The **GloFAS daily discharge file** is the NetCDF file with the daily river discharge from the GloFAS hydrological model for the given period. The code is designed to handle GloFAS hydrological model v4.0, so changes may need to be done for future versions of GloFAS if the file structure changes. To avoid issues, it's better to use the NetCDF file with data from all years aggregated together. And be careful, you can't have this file on NIRD and run it on BETZY/FRAM, you'll need to copy it on your work directory. Also **pay attention to the format**, all the fields that are originally in the GloFAs discharge file must be here with the same name, otherwise you'll have issues. So if you aggregate data from several years using CDO, make sure all the fields are in the aggregated file.
- The **GloFAS LDD file** is the NetCDF file containing the Local Drainage Directions for the GloFAS grid. It is a static auxiliary file, so it doesn't need to be changed as long as the version of the GloFAS hydrological model doesn't change.
- The **GloFAS uparea file** is the NetCDF file containing the Upstream Area for the GloFAS grid. It is a static auxiliary file, so it doesn't need to be changed as long as the version of the GloFAS hydrological model doesn't change.

- The **GloFAS elevation file** is the NetCDF file containing the elevation for the GloFAS grid. It is a static auxiliary file, so it doesn't need to be changed as long as the version of the GloFAS hydrological model doesn't change.
- The **river correction file** is a NetCDF file with the correction parameters for GloFAS data. Details on how to compute this file are given in section 4.
- The **estuaries edit file** is the .csv file containing potential changes in the position of certain GloFAS estuaries. Details on how to compute this file are given in section 3.

2 How to compute high frequency river forcing

This section covers the steps to obtain the river forcing file (rivers.[ab]) from GloFAS data.

To compute the river forcing for a given period, go to *TOPAZ_runoff/scripts/GloFAS*. General instructions are given in the REAMDE that you should find in that folder. Otherwise, the process is as follows:

- First, check the file *compute_discharge_NoUI.py* and look at the list of file paths at the top. These paths lead to the data described in section 1.3. Make sure that the paths are correct.
- Then, in the file *river_forcing.py*, check the path to the input GloFAS dataset (Netcdf file with raw GloFAS discharge data). And choose the start and end date you want for the forcing file (be careful with the format for the date: YYYY-MM-DDT00:00:00.000000000).
- Still in the file *river_forcing.py*, set the variables *Apply_GloFAS_correction* (to correct GloFAS data using correction file, see section 4), *Edit_estuaries* (to change position of certain estuaries using estuaries edit file, see section 3) and *Propagation_cleaning_step* (to enable the last step of river plume propagation, enabling this can create discontinuities in the river forcing grid). These variables must be set to *True* or *False*. The river plume propagation radiuses can also be changed here (value in meters).
- Run file *river_forcing.py*, it may take a while to get going. The forcing .[ab] file will be written in the output folder you indicated.

2.1 Common issues

If the river forcing program doesn't work, it's likely you can solve it by following these steps:

- Check the file paths in *compute_core_discharge_NoUI.py*. You may be using the wrong version for TOPAZ grid files. The GloFAS auxiliary files also have to correspond to the same GloFAS version as the discharge files
- Make sure that the GloFAS discharge file (NetCDF file) is not on NIRD. You have to have it on the same machine you run the program on. Make sure you have the right years in the GloFAS discharge file.
- Check your Python environment to make sure you have all the necessary packages. Issues may occur when opening the GloFAS data netCDF file.
- The problem may occur when the code is trying to read the GloFAS discharge Netcdf file. Depending on the version of GloFAS and modification that have been done to it, the fields in the file may not correspond to the one expected in the program. This will create issues in the *GloFAS_grid.py* file. In particular for the method called *find_estuaries()* where an aggregate function needs to read all the fields. If one is missing, just re-write the definition for the aggregate functions (there are two).

- If the problem persists, you'll have to use a debugger (and somebody who knows Python well) to find the source of the issue and fix it.

3 Computing the estuaries edit file: how to change the position of an estuary

The position of the estuaries on the TOPAZ grid is obtained using a nearest neighbour interpolation on the GloFAS estuaries. This interpolation can sometimes lead to problematic results. So it is possible to manually change the position of an estuary if it is not adequate.

This manual change of estuary location is accomplished using a graphical interface. **You only need to use it once**, with any daily GloFAS file. You can output a CSV file that contains the modifications and use this CSV file whenever creating a river forcing file.

Here is how to edit the position of an estuary:

- Run the file *main.py* in *TOPAZ_runoff/scripts/GloFAS*
- A window called *Parameters* will appear. In this window, you must first give the paths for all the necessary files. These files are the same as those described in section 1.3, **with the exception of the GloFAS discharge file**. This GloFAS discharge file must be the GloFAS discharge file for a specific day in the NetCDF format. Any day will do as long as it is the right version of GloFAS. In fact the discharge values do not matter at all here, since the positions of the estuaries come from the auxiliary files. But this discharge file is required by the program. There you can also change the path to the output files (especially the *Modification file* that will contain the modifications !). The other parameters do not need to be changed. Once everything is set, click on **Run** at the bottom of the window.
- After a little while, a window called *map viewer* will appear. The land is colored by river discharge and the estuaries on the TOPAZ grid are represented as squares. You can load a previous estuary modification file (CSV file with modification to estuary position) by clicking **Load edits**.
- On the map viewer, you can zoom on a particular area using the magnifying glass icon.
- You can select the area containing the estuary you want to edit by clicking and dragging the red rectangle (the magnifying glass icon has to be disabled). Once the area is selected, you can click on **Open editor**.
- A window called *Estuary Editor* will open. There, the estuaries are represented by downward triangle and the TOPAZ grid points by circles. Select the estuary you want to move (triangle), then select the new location (circle) and click on **confirm change**. The estuary should be moved. It can be difficult to select the right point, so you can move and zoom before clicking. Once all is done, click on **Save and exit**.
- This operation can be repeated as many times as possible. If you want to reset the selected area (red area), press *t* twice.
- Once all the modifications are done, you can save them in the modification file by clicking **Save edits** (on the left). If you have loaded modifications, the new ones will be added. The modification file is a CSV file (by default named *estuaries_edit.csv*). You will find it the output path you set in the parameters window. This modification file will be used when creating the river forcing.

- You can also, write all the estuary locations in a text file if necessary, by clicking **Save estuaries** (on the right). They will be written in the Estuaries file (by default named `GloFAS__estuaries.csv`). The path is also set in the parameters window.

All the steps are summed up on Figure 2. In the end, the modification file is the one you will have to use when creating the river forcing. In `compute_core_discharge_NoUI.py`, you can give the path to the modification file in the variable `estuary_edit_file`. You must also set `Edit_estuaries` to `True`

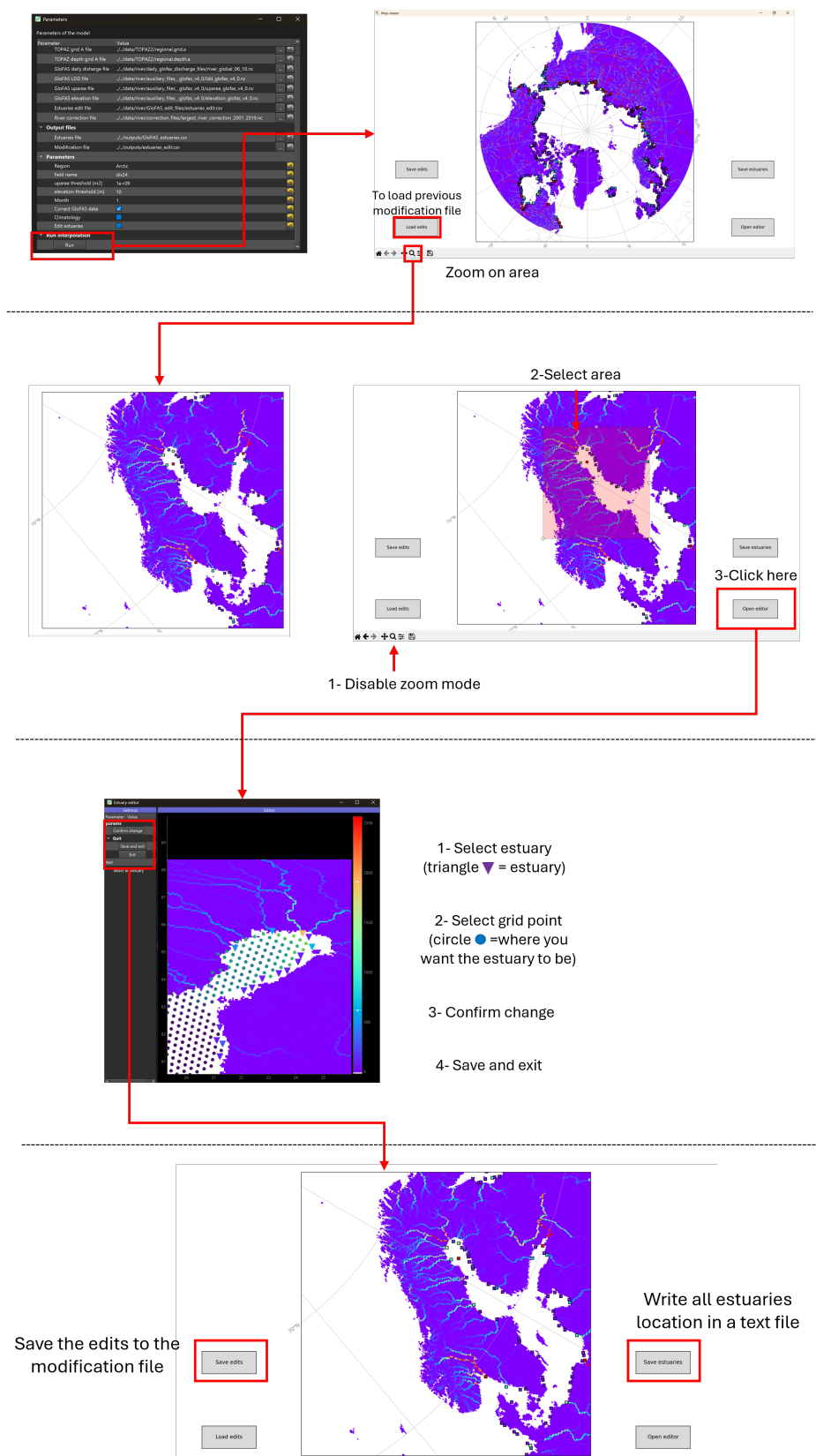


Figure 2 : Changing the position of an estuary

4 Computing the GloFAS correction file

The correction of GloFAS data is done using a NetCDF file containing the correction parameters. This file only needs to be computed once. However, it is important to note that the correction parameters depend on the years used to compute them.

Optimally, you need 20 years of data to compute the GloFAS correction, and it is better to use more recent years. For example, if you run a simulation that goes until 2026, you may want to use years 2001-2025 for the correction.

To compute the correction file, all the necessary programs are in the folder *TOPAZ_runoff/GloFAS_correction_process*. You should find a README there. Otherwise, here are the steps:

- First, you will need to download data from ArcticGRO. The correction is based on this dataset. You need to download discharge data as CSV files from <https://arcticgreatrivers.org/data/> for the 3 rivers (Yenisei, Lena and Ob).
- Then, we will compute Pandas dataframes from these CSV files. This is done using the program *compute_ArcticGRO_Dataframes.py*. You will have to give the path to the folder containing the CSV files in the *path* variable. This should run quite smoothly, but if there is an issue with finding the files, the code is pretty straight forward so feel free to make a few modifications.
- Then you will need the GloFAS data for the years you want to use for the correction. You can get them from someone else or download them from the Copernicus website. Here, it is essential to have data from full years (you can download full years directly from the website), and name them *data_YYYY.nc*.
- Finally, the correction file is computed by running the program *compute_largest_river_correction.py*. At the start of the file, you must give the path to the ArcticGRO dataframes, the GloFAS datasets as well as the start year and end year of the period you want to use to compute the correction. The output file will be called *largest_river_correction_start_end.nc*

These steps are summed up on Figure 3.

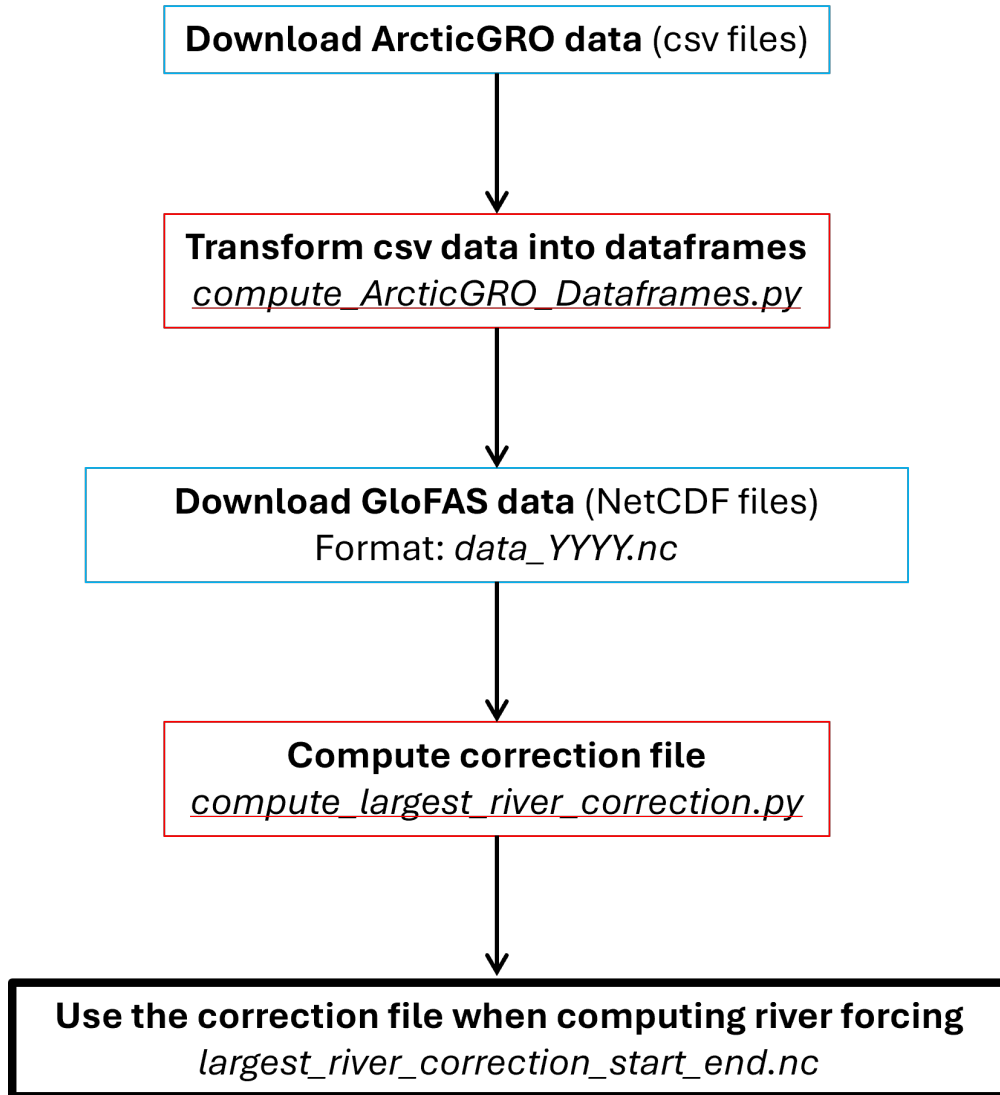


Figure 3 : Steps to compute GloFAS correction file

5 Details on the program

This section goes into further details about the structure of the river forcing program in the folder `TOPAZ_runoff/scripts/GloFAS`.

The code is written in object-oriented way. The main class is *Grids* defined in the file `grids_construction.py`. When initializing a *Grids* object, it will create the different grids for the different types of data (for the TOPAZ grid and the GloFAS grid) using associated classes. Once the grids are created, a series of methods will be run, here is rundown of the actions happening during the initialization of a *Grids* object:

- First, we create *Agrid* objects for the TOPAZ regional grid and the TOPAZ regional depth grid (*Agrid* class is defined in file `Agrid.py`).

- Then, we create a *GloFAS_grid* object. It will contain all the attributes that are on the GloFAS grid (*GloFAS_grid* class is defined in file *GloFAS_grid.py*). And we add the discharge grid to the *GloFAS_grid* object.
- Then we add all the necessary auxiliary GloFAS data (LDD, upstream area, elevation) to the *GloFAS_grid* object. It makes sense because all these variables are on the same grid (the GloFAS grid).
- Using these auxiliary data, we can find the GloFAS estuaries using the method *find_estuaries()* from *GloFAS_grid*.
- Then, we interpolate the GloFAS grid on the TOPAZ grid using method *get_glofas_into_topaz()* from *GloFAS_grid*. For each GloFAS grid cell, this will yield the corresponding (closest) TOPAZ grid cell. So now we have the GloFAS estuaries on the TOPAZ grid (all this information is in the *GloFAS_grid* object which is a variable of the *Grids* object).
- Then if there are any modifications for the estuary positions (see section 3), they are applied using the method *load_modif_file()* from *GloFAS_grid*.
- Next, the GloFAS data correction is applied (see section 4 to know how to compute the correction file). The data is corrected using *correct_largest_russian_rivers()* method from *GloFAS_grid*.
- Finally, the river plume is propagated around the estuaries using *propagate_river_plume()* method from *GloFAS_grid*.

You may notice that there are two ways to go through the list above, depending on the variable named *lazy* (boolean). This variable is here to make sure that we do not re-compute certain things uselessly when creating the river forcing file. Indeed, when we create the river forcing file for several days, we have to run the program for each day. But the only thing that changes from one day to the other is the discharge values. The interpolation, the estuaries locations and the plume propagation weight grids are the same. So to avoid costly and time-consuming computation, when we compute the river forcing, we run the full program only for the first day, and the following days use a shortcut which consists in re-using the grids (*lazy* is set to True).